E-commerce Order Fulfillment Automation Backend

Automate e-commerce order fulfillment with this Java-based console system, managing everything from validation to shipment tracking.

Team Memebers:

1

Saika Sarara

ID: 230042159

2

Adiba Ahsan Raiyan

ID: 230042161

3

Maliha Tasnim Khan

ID: 230042127

Project Overview: Automating the Backend

The E-commerce Order Fulfillment Automation Backend is a console-based system built in Java, focusing on automating internal order processing. It handles operations after a customer places an order, ensuring a seamless backend workflow.

Core Processes

- Order validation
- Inventory verification & reservation
- Invoice generation
- Payment simulation
- Shipment creation & tracking
- Notification handling
- Workflow logging & reporting

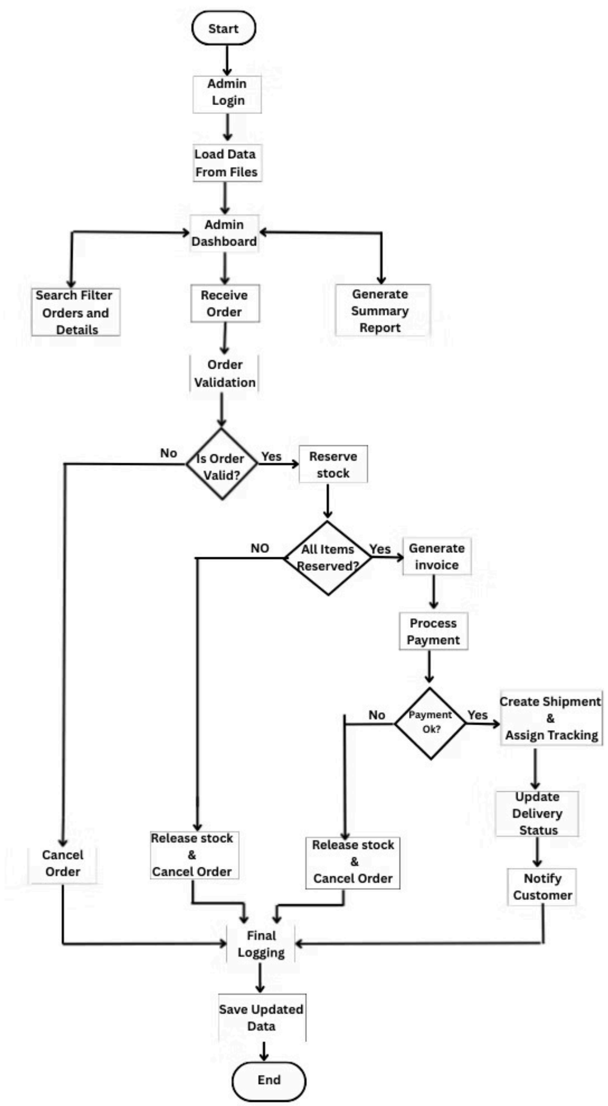
Motivation Behind the Project

During the festival season, we noticed a small online shop in Bangladesh repeatedly getting our orders wrong. Due to the absence of any automated system, their team often mismatched stock, missed invoices, or shipped products late. As customers, this was frustrating and clearly showed that the issue was not with the products, but with the workflow.

That observation led to the idea for the **E-commerce Order Fulfillment Automation Backend.**

Instead of building another customer interface, we chose to focus on the backstage of an order — the validation, stock handling, invoicing, payment checks, shipping, and logging that real businesses depend on. Our goal was to turn the invisible journey of an order into a structured, automated process that can be clearly designed, executed, and analyzed using clean backend logic.

Flow Chart



Start
Admin Login
Load Data From Files
Admin Dashboard
Search Filter Orders and Details
Receive Order
Generate Summary Report
Order Validation
Is Order Valid?
No
Yes
Reserve stock
All Items Reserved?
NO
Yes
Generate invoice
Process Payment
Payment Ok?
No
Yes
Create Shipment & Assign Tracking
Update Delivery Status
Notify Customer
Cancel Order
Release stock & Cancel Order
Release stock & Cancel Order
Final Logging
Save Updated Data
End

Key Features

1. Secure Admin Login

Admin-only access with hashed passwords (not plain text) prevents unauthorized backend access.

2. Order Intake

Orders loaded from JSON files with required fields (orderId, items[], address, paymentMode), queued for processing.

3. Order Validation

Checks valid structure, non-empty items, positive quantities; invalid orders cancelled and logged.

4. Inventory Verification & Reservation

Verifies stock availability, uses whole-order rejection if unavailable, reserves on success or releases and cancels on failure.

Key Features

5. Admin Dashboard

Menu-driven CLI dashboard with order viewing, search/filter by ID or status, access to details, invoices, tracking, and logs.

6. Manual Status Control

Admin can manually update shipment status, system prevents invalid transitions, every update logged for audit.

7. Workflow Logging & Audit Trail

Logs each step with timestamp, orderId, step name, outcome/message; stored per order and in master log file.

8. Reporting System

Auto-generates reports showing total orders, successful vs cancelled, cancellation reasons, total revenue; stored as files.

Key Features

9. Workflow Logging & Audit Trail

Logs each step with timestamp, orderId, step name, outcome/message; stored per order and in master log file.

10. Reporting System

Auto-generates reports showing total orders, successful vs cancelled, cancellation reasons, total revenue; stored as files.

11. Data Persistence

File-based storage (products.json, orders.json, customers.json, invoices.json), serialized/deserialized, ensures data availability across runs.

12. Test Scenarios

Includes seeded test data covering successful order flow, invalid order, stock shortage, payment failure.

Tools & Technology

Programming Language

Java

Execution Environment

Console-based (CLI) application

Version Control

Git & GitHub

Storage

File-based JSON / structured text

Architecture

Modular, Object-Oriented, workflow-based design

Libraries

Minimal lightweight utilities (e.g., JSON parsing library)

Project Timeline

Weeks 1–4: Proposal Phase

Week 1: Problem identification & requirement analysis
Week 2: Feature planning & scoping
Week 3: Flowchart & UML design
Week 4: Draft proposal & presentation

Weeks 5–6: Proposal Finalization

Week 5: Feedback incorporation & refinement
Week 6: Final proposal submission

Weeks 7–9: Core Implementation

Week 7: Core entity classes
Week 8: Order validation & inventory logic
Week 9: Stock reservation & cancellation

Weeks 10–11: Mid-Semester Work

Week 10: Invoice generation & payment simulation
Week 11: Shipping workflow & tracking

Week 12: Progress Presentation

Progress presentation & feedback incorporation

Weeks 13–14: Advanced Features + Early Testing

Week 13: Admin dashboard, search, manual status updates
Week 14: Data persistence, logging, initial testing

Week 15: Security + Final Testing

Password hashing, bug fixing, summary report generation

Week 16: Final Light Week

Documentation finalization, final presentation & submission

Thank You

We appreciate your time and attention. We are excited about the future of automated e-commerce fulfillment.